

JavaScript Variables — Revision Notes

Interview + practical revision

1. What is a variable?

A variable is a named binding to a value.

```
let age = 26;
let name = "Vikrant";
let isDeveloper = true;
```

2. var, let, and const

Modern JavaScript mainly uses **let** and **const**.

	var	let	const
Reassign	Yes	Yes	No
Redeclare same scope	Yes	No	No
Block scoped	No	Yes	Yes
Hoisted	Yes	Yes*	Yes*
Recommended	Generally avoid	Yes	Yes

Rule of thumb: const → default choice; let → when the value needs to change; var → generally avoid.

3. const doesn't mean immutable

```
const user = { name: "John" };

user.name = "Mike"; // works
user = {}; // TypeError
```

const prevents **reassignment of the binding**, not modification of the object or array.

4. Scope

let and const are **block scoped**. var is **function scoped**.

```
if (true) {
  let x = 10;
}
console.log(x); // ReferenceError
```

```
if (true) {
  var x = 10;
}
console.log(x); // 10
```

5. Shadowing

A variable inside a block can have the same name as an outer variable. The inner variable shadows the outer one.

```
let name = "John";
```

```
{
  let name = "Mike";
  console.log(name); // Mike
}

console.log(name); // John
```

6. Hoisting

var: the declaration is hoisted and initialized with undefined.

```
console.log(x);
var x = 10;

// undefined
```

let and const: declarations are hoisted, but cannot be accessed before their declaration because of the **Temporal Dead Zone (TDZ)**.

```
console.log(x);
let x = 10;

// ReferenceError
```

Remember: Hoisted does not mean accessible.

7. undefined vs undeclared

```
let x;
console.log(x); // undefined
```

The variable exists, but no value has been assigned.

```
console.log(y); // ReferenceError
```

Here, y has not been declared.

8. Variable naming

```
let userName;
let _count;
let $price;
let age2;
```

JavaScript is case-sensitive. Names cannot start with a number and cannot use reserved keywords.

```
let age = 20;
let Age = 30;

console.log(age); // 20
console.log(Age); // 30
```

9. Multiple declarations

```
let a = 10, b = 20, c = 30;
```

Valid, although separate declarations are often easier to read.

10. Interview traps

Question 1

```
console.log(a);  
var a = 10;
```

Question 2

```
console.log(a);  
let a = 10;
```

Question 3

```
const user = { name: "John" };  
user.name = "Mike";  
console.log(user.name);
```

Question 4

```
let x = 10;  
{  
  let x = 20;  
  console.log(x);  
}  
console.log(x);
```

Question 5

```
if (true) {  
  var x = 10;  
}  
console.log(x);
```

Question 6

```
var x = 1;  
{  
  var x = 2;  
}  
console.log(x);
```

Quick mental model

1. const → cannot reassign
2. let → can reassign
3. var → function scoped, avoid in modern JS
4. let/const → block scoped
5. let/const → TDZ before declaration

Recommended revision order: Variables → Scope → Hoisting → Closures